

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_09101709-554\agent-  
a0760fcfdbf29c7f0.jsonl`  
- SHA-256: `86adea8511f9b28eb92a829260cb8961b63187217672d0fa1f1ed2026459cda3`  
- Source modified: `2026-06-25T17:13:08+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_09101709-554`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:11:13.830Z)

PR #381 ".github/workflows/nightly.yml" is CHECKED OUT (branch chore/nightly-quality-regression). It adds a Tier-1 nightly quality-regression GitHub Actions workflow (schedule cron "0 3 * * *" + workflow_dispatch) that re-runs the repo's OFFLINE guards (ruff, mypy, run_all_tests --cov-fail-under=70, golden-fixtures regression). The ONLY changed file is .github/workflows/nightly.yml (105 lines, ci.yml untouched). CRITICAL CONTEXT: a schedule/workflow_dispatch workflow does NOT self-trigger on the PR, so the PR's green CI only YAML-PARSED nightly.yml ? its STEPS were never executed. So you must verify the steps would actually run. Read BOTH .github/workflows/nightly.yml AND .github/workflows/ci.yml to compare. Ground every claim in the files.

Review ONLY security + robustness of the workflow. (a) permissions: ? is it least-privilege (contents: read), or does it inherit broad defaults? (b) Any pull_request_target / untrusted-input / checkout-of-untrusted-ref footgun? (does it need any secrets?) (c) concurrency: group to avoid overlapping nightly runs? job timeout-minutes so a hung run doesn't burn minutes? (d) failure visibility ? a nightly that fails silently is useless; is there any notification/surfacing, or at least is failure observable? Report findings + fix. Don't invent issues ? a simple offline test runner is low-risk; only flag real gaps.

2. assistant (2026-06-25T17:11:16.552Z)

I'll review PR #381's nightly.yml workflow for security and robustness. Let me start by reading both workflow files.

3. user (2026-06-25T17:11:17.012Z)

```
1      name: Nightly quality regression (Tier-1, offline)  
2  
3      # Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;  
4      # this runs the SAME offline guards on a schedule so we also catch what per-PR CI  
5      # can't: dependency / transitive-dep drift, flakiness over time, and rot during  
6      # quiet periods when nothing is pushed.  
7      #  
8      # Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the  
9      # real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +  
10     # the gitignored golden corpus). The lanes here mirror `ci.yml`'s offline jobs  
11     # verbatim (same Python pin, same per-lane pip installs) so modules collect/run  
12     # instead of silently skipping for a missing dep.  
13     #  
14     # NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this  
15     # won't actually run on a cadence until it lands on `master`. Use the  
16     # `workflow_dispatch` trigger to run it on demand for validation after merge.  
17  
18     on:  
19         schedule:
```

```

20     # 03:00 UTC daily.
21     - cron: "0 3 * * *"
22     workflow_dispatch:
23
24     permissions:
25         contents: read
26
27     concurrency:
28         group: nightly-`${{ github.ref }}`
29         cancel-in-progress: true
30
31     jobs:
32         # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
33         lint:
34             name: Ruff lint
35             runs-on: ubuntu-latest
36             steps:
37                 - uses: actions/checkout@v6
38                 - uses: actions/setup-python@v6
39                   with:
40                       python-version: "3.13"
41                 - name: Install ruff
42                   run: python -m pip install ruff
43                 - name: Ruff check
44                   run: ruff check . --output-format github
45
46         # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
47         # deps are installed so mypy sees real signatures (without them
48         # ignore_missing_imports would Any-stub half the checks).
49         typecheck:
50             name: Mypy (lenient baseline)
51             runs-on: ubuntu-latest
52             steps:
53                 - uses: actions/checkout@v6
54                 - uses: actions/setup-python@v6
55                   with:
56                       python-version: "3.13"
57                 - name: Install mypy + typed core deps
58                   run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
59 genai numpy
60                 - name: Mypy
61                   run: mypy backend training
62
63         # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
64         # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
65         # rest. Floor stays at the ci.yml value (70); never lower it without an owner
66         # decision.
67         full-suite:
68             name: Full test suite (offline, mocked)
69             runs-on: ubuntu-latest
70             steps:
71                 - uses: actions/checkout@v6
72                 - uses: actions/setup-python@v6
73                   with:
74                       python-version: "3.13"
75                       cache: pip
76                 - name: System libs for opencv-python
77                   run: |
78                       sudo apt-get update
79                       sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
80 libgl1 libglib2.0-0
81                 - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
82                   run: python -m pip install torch torchvision --index-url
83 https://download.pytorch.org/whl/cpu
84                 - name: Install package (editable) + dev tooling

```

```

82         run: python -m pip install -e .[dev]
83     - name: Run full suite (with coverage floor)
84         run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
85
86     # Offline quality guard: the committed golden regression fixtures
87     # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
88     # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
89     # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
90     # every PR ? nightly is about drift-over-time on the default branch). The
91     # CONTROL replay needs only numpy.
92     golden-fixtures:
93         name: Golden fixtures regression (offline quality guard)
94         runs-on: ubuntu-latest
95         steps:
96             - uses: actions/checkout@v6
97             - uses: actions/setup-python@v6
98               with:
99                 python-version: "3.13"
100             - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
101               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
102         - name: Golden-fixtures characterization (offline)
103           run: |
104             python -m backend.eval.golden_fixtures --check
105             python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
106

```

4. user (2026-06-25T17:11:17.532Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only
19         # py_compile smoke sweep does not reach.
20         lint:
21             name: Ruff lint
22             runs-on: ubuntu-latest
23             steps:
24                 - uses: actions/checkout@v6
25                 - uses: actions/setup-python@v6
26                   with:
27                     python-version: "3.13"
28                 - name: Install ruff
29                   run: python -m pip install ruff
30                 - name: Ruff check
31                   run: ruff check . --output-format github
32
33         # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
34         # (the video_id shape) or a personal absolute user path in the shipped-code surface

```

```

35     # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures).
Real
36     # names are scanned only via a gitignored private list locally; CI runs the structural
guard.
37     leak-scan:
38         name: Leak scan (PII / attribution guard)
39         runs-on: ubuntu-latest
40         steps:
41             - uses: actions/checkout@v6
42             - uses: actions/setup-python@v6
43               with:
44                 python-version: "3.13"
45             - name: Leak scan
46               run: python -m tools.leak_scan
47
48     # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
49     # per-module quarantine list ? ratchet by removing entries as modules clean
50     # up. Typed core deps are installed so mypy sees their real signatures
51     # (without them ignore_missing_imports would Any-stub half the checks).
52     typecheck:
53         name: Mypy (lenient baseline)
54         runs-on: ubuntu-latest
55         steps:
56             - uses: actions/checkout@v6
57             - uses: actions/setup-python@v6
58               with:
59                 python-version: "3.13"
60             - name: Install mypy + typed core deps
61               run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
62             - name: Mypy
63               run: mypy backend training
64
65     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
66     # module-level import error shipping while the suite stays green.
67     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
68     # on machines without the GPU stack (test_import_smoke stubs what's absent).
69     import-smoke:
70         name: Syntax sweep + import smoke (no GPU stack)
71         runs-on: ubuntu-latest
72         steps:
73             - uses: actions/checkout@v6
74             - uses: actions/setup-python@v6
75               with:
76                 python-version: "3.13"
77             - name: Install minimal deps
78               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
79             - name: Run import smoke tests
80               run: python -m pytest tests/test_import_smoke.py -v
81
82     full-suite:
83         name: Full test suite (offline, mocked)
84         runs-on: ubuntu-latest
85         steps:
86             - uses: actions/checkout@v6
87             - uses: actions/setup-python@v6
88               with:
89                 python-version: "3.13"
90                 cache: pip
91             - name: System libs for opencv-python
92               run: |
93                 sudo apt-get update
94                 sudo apt-get install -y libgl1 libgl2.0-0 || sudo apt-get install -y
libgl1 libgl2.0-0

```

```

95     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
96     # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
97     # need an out-of-band index that PEP 621 can't express. Keep installing
98     # them separately here, before the editable install pulls the rest.
99     run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
100     - name: Install package (editable) + dev tooling
101     # Editable PEP 621 install: runtime deps + the `dev` group
102     # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
103     run: python -m pip install -e .[dev]
104     # obsreport (private, soft dep) is absent here: reporting tests skip via
105     # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
106     # is therefore calibrated against THIS lane's number (local runs with
107     # obsreport installed measure a little higher).
108     # Floor calibrated 2026-06-11 against this lane's measured 72% (local
109     # with obsreport: 73%). Ratchet it UP as coverage grows; never down
110     # without an owner decision.
111     - name: Run full suite (with coverage floor)
112     run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
113
114     # Cross-OS determinism guard for the committed golden regression fixtures
115     # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
116     # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
117     # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
118     # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
119     # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
120     golden-crossos:
121       name: Golden fixtures cross-OS (${{ matrix.os }})
122       strategy:
123         fail-fast: false
124         matrix:
125           os: [ubuntu-latest, windows-latest, macos-latest]
126       runs-on: ${{ matrix.os }}
127       steps:
128         - uses: actions/checkout@v6
129         - uses: actions/setup-python@v6
130         with:
131           python-version: "3.13"
132         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
133         run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
134         - name: Golden-fixtures characterization (cross-OS determinism guard)
135         # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
136         # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
137         # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
138         run: |
139           python -m backend.eval.golden_fixtures --check
140           python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
141
142     # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
143     # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
144     # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
145     # automates a guardrail that was human-only; the report is printed for review either
way.
146     license-scan:
147       name: Dependency license scan (no copyleft in runtime deps)
148       runs-on: ubuntu-latest

```

```

149     steps:
150     - uses: actions/checkout@v6
151     - uses: actions/setup-python@v6
152       with:
153         python-version: "3.13"
154     - name: Install runtime deps + pip-licenses
155       run: |
156         python -m pip install .
157         python -m pip install pip-licenses
158     - name: Report dependency licenses
159       run: python -m piplicenses --format=markdown --with-urls --order=license
160     - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
161       run: |
162         if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
163           echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
164           exit 1
165         fi
166         echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."
167
168     # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
169     # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
170     # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
171     # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
172     build-artifact:
173       name: Built wheel smoke (install + launch)
174       runs-on: ubuntu-latest
175       steps:
176       - uses: actions/checkout@v6
177       - uses: actions/setup-python@v6
178         with:
179           python-version: "3.13"
180       - name: System libs for opencv-python
181         run: |
182           sudo apt-get update
183           sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
184       - name: Build the wheel
185         run: |
186           python -m pip install build
187           python -m build --wheel
188       - name: Install the built wheel + CPU torch (out-of-band index)
189         run: |
190           python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
191           python -m pip install dist/*.whl
192       - name: Launch the installed console script from a non-repo dir
193         run: |
194           cd /tmp
195           indexer --help
196           python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
197

```

5. assistant (2026-06-25T17:11:20.378Z)

Both files are read. Now let me verify the claims in the workflow comments against the actual repo to ground my findings ? checking that the referenced modules/test files and the `run\_all\_tests.py` flags actually exist.

6. user (2026-06-25T17:11:22.999Z)

```

-rwxr-xr-x 1 avidu 197609 2055 Jun 7 14:52 run_all_tests.py
-rw-r--r-- 1 avidu 197609 7293 Jun 25 16:24 tests/test_golden_fixtures.py
-rw-r--r-- 1 avidu 197609 28710 Jun 25 16:24 tests/test_golden_real_fixtures.py
---PYPROJECT dev extra---
10:# which PEP 621 cannot express. Install them separately ? see [project.optional-dependencies]
11-# `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12-[build-system]
13-requires = ["hatchling"]
14-build-backend = "hatchling.build"
15-
16-[project]
17-name = "badminton-highlight-indexer"
18-version = "0.1.0"
--
43:[project.optional-dependencies]
44-# Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
45:dev = [
46-    "pytest",
47-    "pytest-cov",
48-    "httpx",          # fastapi TestClient transport
49-    "ruff",
50-    "mypy",
51-    "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
52-]
53-

```

7. user (2026-06-25T17:11:24.665Z)

```

1a400c0 ci: add Tier-1 nightly quality-regression workflow
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
71a8241 chore(repo): fix corrupted .gitignore (UTF-16 tail) + ignore artifacts/, track product
overview (#377)
f7fb36e docs(audit): fix two accuracy nits ? D2 DoD wording + main.py line count (#376)
---branch---
chore/nightly-quality-regression
---diff vs master---
.github/workflows/nightly.yml | 105 ++++++++++++++++++++++++++++++++++++++
1 file changed, 105 insertions(+)

```

8. assistant (2026-06-25T17:11:28.211Z)

The scope is confirmed: only `nightly.yml` added (105 lines), referenced test files and the `dev` extra all exist. Let me verify the `golden\_fixtures --check` module entrypoint and confirm `run\_all\_tests.py` accepts the passthrough flags, then check whether the repo already has any notification/surfacing pattern for failures (e.g., issue-on-failure, Slack) elsewhere.

9. user (2026-06-25T17:11:29.874Z)

```

#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```

python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage

```

Anything after the script name is passed straight through to pytest.

Notes

- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional `obsreport` package isn't installed (they use `pytest.importorskip`). To exercise them too:
`pip install -e ../sports-obsreport --no-build-isolation`
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).

```

"""
from __future__ import annotations

import importlib.util
import subprocess
import sys

def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False

def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests will
SKIP"
    bar = "=" * 72
    print(bar)
    print(" Full test suite: " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)

    rc = subprocess.run(cmd).returncode

    print(bar)
    print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}"))
    print(bar)
    return rc

if __name__ == "__main__":
    raise SystemExit(main())
=== golden_fixtures __main__ ===
backend/eval/golden_fixtures.py

```

10. user (2026-06-25T17:11:31.979Z)

```

=== other workflows ===
ci.yml
nightly.yml
=== any failure-notify / issue-on-fail / slack patterns anywhere ===
(end)

```

11. assistant (2026-06-25T17:11:35.162Z)

Confirmed: `run_all_tests.py` passes everything through to pytest (so `--cov-fail-under=70` works), `golden_fixtures.py` exists with a `--check` entrypoint, and there is **no** existing failure-notification pattern anywhere in `.github/`. Let me verify the `golden_fixtures` module actually supports `--check` (argparse) so the nightly step won't error.

12. user (2026-06-25T17:11:36.103Z)

```

52:import argparse
727: ap = argparse.ArgumentParser(
730: g.add_argument("--freeze-synthetic", action="store_true",
733: g.add_argument("--refresh-snapshot", action="store_true",
735: g.add_argument("--check", action="store_true",
737: g.add_argument("--freeze-real", action="store_true",
740: ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
742: ap.add_argument("--all", action="store_true", help="all slugs for --refresh-snapshot")
743: ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
745: ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
746: ap.add_argument("--labels", help="--freeze-real) source golden labels CSV (start,end)")
747: ap.add_argument("--license", dest="license_", default=None,
749: ap.add_argument("--owned", action="store_true",
751: ap.add_argument("--minors-free", dest="minors_free", action="store_true",
753: ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
754: ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-real)
source frame width (px)")
755: ap.add_argument("--label", default="cleared_real",
758: ap.add_argument("--source-note", dest="source_note", default="",
762: ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
764: ap.add_argument("--max-frames", dest="max_frames", type=int, default=None,

```

13. assistant (2026-06-25T17:11:49.878Z)

All claims verified against the code. The steps mirror `ci.yml` faithfully and reference real entrypoints. Now I have everything needed to assess security + robustness.

Let me summarize the grounded analysis against the four review axes:

****(a) permissions**** ? `nightly.yml` lines 24-25 set `permissions: contents: read`, matching `ci.yml` lines 8-9. Least-privilege, explicit, does NOT inherit broad defaults. Good.

****(b) untrusted-input footgun**** ? Trigger is `schedule` + `workflow\_dispatch` only (lines 18-22). No `pull\_request\_target`, no `workflow\_run`, no fork PR. Checkout is `actions/checkout@v6` with no `ref:` override ? checks out the default branch SHA the schedule fires on. No secrets referenced or needed (all offline guards). No injection of untrusted context into `run:` blocks. Clean.

****(c) concurrency + timeout**** ? `concurrency.group: nightly-\${{ github.ref }}` + `cancel-in-progress: true` (lines 27-29) prevents overlapping runs on the same ref. ****Gap:** no `timeout-minutes` on any of the 4 jobs. ****** A hung step (e.g., pip resolver stall, apt hang) rides GitHub's default 6-hour job timeout, burning Actions minutes on a job that runs unattended nightly. `ci.yml` also lacks this, but a scheduled unattended job is exactly where a hang is invisible and costly ? this is the strongest robustness gap.

****(d) failure visibility**** ? This is the ****core purpose-defeating gap****. The workflow header (lines 3-6) states the whole point is to "catch what per-PR CI can't... rot during quiet periods when nothing is pushed." But a scheduled run with no PR/push has no surface: nobody is watching the Actions tab on a quiet day. There is no `if: failure()` notification step, no issue-on-failure, no Slack/email beyond GitHub's default (default scheduled-failure emails go only to the workflow's last editor, and only after consecutive runs / can be missed). A nightly that fails silently is exactly the dead-canary problem the doc warns about. No existing pattern in `.github/` to follow, confirming this is net-new.

Let me write the fix for the two real gaps (timeout-minutes + failure surfacing).

14. user (2026-06-25T17:11:55.939Z)

Noted (position 1, task_id: task_4d164c93). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_4d164c93 "Add timeout-

minutes to ci.yml jobs". Continue your current work.

15. assistant (2026-06-25T17:11:58.950Z)

Now I'll provide the fixed `nightly.yml` as my recommended patch (timeout-minutes on each job + a failure-surfacing job). Since this PR is checked out, let me apply the robustness fixes directly so the user has a concrete diff.

16. user (2026-06-25T17:12:00.867Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml has been updated successfully. (file state is current in your context ? no need to Read it back)

17. user (2026-06-25T17:12:02.749Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml has been updated successfully. (file state is current in your context ? no need to Read it back)

18. user (2026-06-25T17:12:04.665Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml has been updated successfully. (file state is current in your context ? no need to Read it back)

19. user (2026-06-25T17:12:06.190Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml has been updated successfully. (file state is current in your context ? no need to Read it back)

20. assistant (2026-06-25T17:12:10.475Z)

Now let me add a failure-surfacing job. The cleanest no-secrets approach is an `if: failure()` job that opens/updates a tracking issue via `actions/github-script` (uses the built-in `GITHUB\_TOKEN`). That requires `issues: write` permission scoped to that job. Let me add it.

21. user (2026-06-25T17:12:24.572Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml has been updated successfully. (file state is current in your context ? no need to Read it back)

22. assistant (2026-06-25T17:12:30.715Z)

Let me validate the final YAML parses correctly (the PR's green CI never executed these steps, and I've added a non-trivial `github-script` block).

23. user (2026-06-25T17:12:33.168Z)

YAML OK

```
jobs: ['lint', 'typecheck', 'full-suite', 'golden-fixtures', 'notify-on-failure']
top perms: {'contents': 'read'}
notify perms: {'contents': 'read', 'issues': 'write'}
notify if: failure() && github.event_name == 'schedule'
timeouts: {'lint': 10, 'typecheck': 10, 'full-suite': 30, 'golden-fixtures': 15, 'notify-on-failure': 5}
```

24. assistant (2026-06-25T17:12:35.881Z)

YAML parses cleanly, all five jobs present, top-level permissions stay `contents: read`, the notify job scopes `issues: write` to itself, and timeouts are set on every job. The fixes are validated. Let me submit the findings.

25. user (2026-06-25T17:12:54.929Z)

Structured output provided successfully

26. assistant (2026-06-25T17:13:08.268Z)

I have completed the security + robustness review of PR #381's `nightly.yml` and applied the fixes. Here is the summary.

What I verified (grounded in the files)

Read both `.github/workflows/nightly.yml` and `.github/workflows/ci.yml`. Confirmed against the repo that every claim the workflow makes is real: `run\_all\_tests.py` passes flags straight through to pytest (so `--cov-fail-under=70` works), `backend/eval/golden\_fixtures.py` exists and supports `--check` (argparse, line 735), both `tests/test\_golden\_fixtures.py` and `tests/test\_golden\_real\_fixtures.py` exist, and the `dev` extra in `pyproject.toml` provides the test/lint/type tooling. The four guard lanes mirror `ci.yml`'s offline jobs faithfully.

Verdict on the four review axes

- **(a) permissions** ? Already correct. `permissions: contents: read` (lines 24-25), explicit and least-privilege, matching `ci.yml`. No change needed.
- **(b) untrusted input / secrets** ? Clean. Triggers are `schedule` + `workflow\_dispatch` only; no `pull\_request\_target`, no `workflow\_run`, no fork-PR exposure. `actions/checkout@v6` with no `ref:` override checks out the default-branch SHA. No secrets referenced or needed (all offline). No injection of untrusted context into `run:` blocks.
- **(c) concurrency + timeout** ? Concurrency group is present and correct. **Timeout was missing on every job** ? fixed.
- **(d) failure visibility** ? **The core gap.** A scheduled run had no surface at all ? fixed.

Fixes applied to `nightly.yml` (validated: YAML parses, 5 jobs, perms correct)

1. Added `timeout-minutes` to each job (lint 10, typecheck 10, full-suite 30, golden-fixtures 15).
2. Added a `notify-on-failure` job (`if: failure() && github.event\_name == 'schedule'`) that opens or refreshes a single `nightly-failure`-labelled tracking issue via the built-in `GITHUB\_TOKEN` (no secrets), with `issues: write` scoped to that job only ? the guard jobs keep `contents: read`.

I also spawned a follow-up chip to add matching `timeout-minutes` to `ci.yml`'s 7 jobs (same omission there), and flagged one NIT for owner awareness: `cancel-in-progress: true` means a cancelled run won't trip the new notifier ? acceptable, but `false` (queue instead of cancel) is an option if you want every started nightly to complete.

Modified file: `C:\Users\avidu\Projects\badminton-highlight-indexer\.github\workflows\nightly.yml`